

Class 2

Managing Processes

THE SHAPE OF THIS COURSE

- Class 1: services already exist, and **systemctl** starts and stops them
- **Today: you build the thing systemd manages**
- Class 3 onwards: you deploy a web server, a database, containers — and every one of them is a process that has to survive you logging out

Today is the bridge. Everything after this assumes it.

Why can't I just run it and go home?

THE QUESTION

```
$ ./myapp &  
$ exit  
  
# ... come back tomorrow. Is it running?
```

- **The answer is no — and the reason is the whole of today**
- By the end you will have turned this into a service that survives a logout, restarts itself when it crashes, and comes back after a reboot

1

Seeing what is running

You cannot manage what you cannot see — and `ps` shows you far less than you think

`ps` does not show you the system

PART 1 · SEEING

```
$ ps                # only YOUR terminal – your shell, and ps itself
$ ps aux            # everything. BSD style, with %CPU and %MEM
$ ps -ef            # everything. UNIX style, with the PPID column
$ ps -o pid,ppid,user,stat,cmd -p 1234
```

- Two or three lines on a machine running well over a hundred processes
- **"I ran ps and my service wasn't there" is not evidence of anything**

PART 1 · SEEING

```
UID      PID      PPID  C  STIME TTY          TIME CMD
root      1         0   0  Aug05 ?           00:00:26 /sbin/init
    ^^^      ^^^^
    |         parent – who started it
    |         this process
```

- **pstree -p** draws the same processes as the tree they actually form
- **Every process on the machine descends from pid 1. You met it in lab01; today it starts mattering.**

Finding one process, and whose it is

PART 1 · SEEING

```
$ pgrep -af myapp      # -f matches the whole COMMAND LINE
$ pgrep -u sysadmin    # everything owned by one user
$ ps -o user,pid,cmd -C nginx
```

- **-f matters: without it pgrep matches only the process NAME — and for a shell script that is the interpreter, not your script. You will hit this today.**
- A service usually runs as its own dedicated user, not as you and not as root
- Check it in class 3 that user is why your files are unreadable

`top`, and which ports are open

PART 1 · SEEING

```
$ top      # q to quit. htop is the same idea, in colour
$ ss -tln  # which ports are LISTENING

LISTEN 0  4096  0.0.0.0:22  0.0.0.0:*
```

- The load average is three figures over three different windows, shortest first. `top`'s first line, every time.
- From class 3, **`ss -tln` is how you answer "is my server actually listening?" — and very often the answer is no**

Add `p` for the owning process — and find the column mostly empty. Seeing who owns a port you do not own needs root.

2

Your terminal owns your processes

Close the terminal and your program goes with it.

Jobs belong to one shell

PART 2 · DETACHING

```
$ ./heartbeat.sh &      # background THIS shell
$ jobs                  # [1] – job numbers, not pids
$ fg                    # bring it back
^Z                      # suspend it where it stands
$ bg                    # let it run again, in the background
$ kill %1               # by job number
```

- CTRL+C asks it to stop. CTRL+Z freezes it instead — a stopped process is still there, using nothing
- **Job numbers exist only inside that one shell. Nobody else can see them.**

Then you log out

PART 2 · DETACHING

```
$ ./heartbeat.sh &  
$ exit  
  
$ shopt -s huponexit      # this shell, hang up on your jobs  
$ nohup ./heartbeat.sh &  # ignore the hangup; log somewhere
```

- **SIGHUP** is what a terminal sends when it goes away. Bash passes it on to its own jobs when **bash** itself is hung up — and **huponexit** decides whether a polite exit does the same.
- **nohup** does two separate things: the program ignores **SIGHUP**, and its output gets somewhere to land that is not your terminal.

A program whose terminal has gone still has a stdout. You can ask any process of your own where its own points:
`readlink /proc/<pid>/fd/1`

PART 2 · DETACHING

- Start it with `&`, no redirect. Log out. Log back in.
- Hands up: still running? And where have its heartbeats been going?
- Run it — then stop, and leave them to check the second half in the lab.
- Everyone has had a job die when they closed a window, and everyone has had one survive. Both are true, and the difference is not luck.

3

Signals

`kill` does not kill. It asks — and a program is free to say no.

`kill` sends a message

PART 3 · SIGNALS

SIGHUP	1	your terminal went away
SIGINT	2	CTRL+C – please stop
SIGKILL	9	stop, now, by the kernel
SIGTERM	15	please shut down cleanly

- `kill <pid>` sends the default one. `kill -9 <pid>` names signal 9.
- **One of these four cannot be caught, blocked or ignored — the kernel delivers it and the program never gets a say. Work out which in the lab.**

`kill -l` lists all sixty-four. `man kill` says which one is the default.

Why `systemctl stop` sometimes waits

PART 3 · SIGNALS

```
trap 'echo "caught it – not going anywhere"' TERM
```

- A real service traps the polite signal so it can finish writing to disk before it exits. That is correct behaviour, not a bug.
- So `systemd` asks politely, waits — up to 90 seconds — and only then uses the one that cannot be refused.
- Which is why reaching for `kill <...>` first is a bad habit: nothing gets closed, nothing gets flushed, and a database can be left mid-write.

4

Hand it to systemd

"It's running" is not "it's deployed".

What `nohup` still cannot do

PART 4 · DEPLOYING

- It does not survive a reboot. You will reboot in the lab — decide now what you expect to find, then go and look.
- Nothing restarts it if it crashes at three in the morning.
- Nobody but you knows it is supposed to exist, or how to start it again.
- **A unit file fixes all three at once — and it is nine lines.**

PART 4 · DEPLOYING

```
/etc/systemd/system/int134-heartbeat.service
```

[Unit]	Description=	what status shows first
[Service]	ExecStart=	the command. ABSOLUTE PATH
	User=	which account. Without it: root
	Restart=	what to do when it stops
[Install]	WantedBy=	what `enable` hooks it to

- **ExecStart** is where first unit files go wrong.
- **systemd** does not run your script from a shell: no ~, no working directory, no relative path

enable is not start — again

PART 4 · DEPLOYING

```
$ sudo systemctl daemon-reload          # systemd re-reads the file
$ sudo systemctl enable --now int134-heartbeat
$ systemctl is-enabled int134-heartbeat # at boot?
$ systemctl is-active int134-heartbeat # right now?
```

- `systemd` read your unit file once and kept it. Edit the file and it will tell you the two have drifted apart — read the warning, it names the fix.
- `enable` writes one symlink into `multi-user.target.wants`. That `symlink` is the entire meaning of "enabled", and it is what `[Install]` was for.

enable is not start — again

THE POINT OF THE WHOLE CLASS

```
$ systemctl status int134-heartbeat
Main PID: 55433 (heartbeat.sh)

$ sudo kill -<..> 55433          # the signal that cannot be refused

$ systemctl status int134-heartbeat
Main PID: ...                  # look carefully
```

- The service is not the process. Watch the Main PID and you will see why that sentence matters.

Where did the output go?

PART 4 · DEPLOYING

```
$ journalctl -u int134-heartbeat -n 20    # last 20 lines
$ journalctl -u int134-heartbeat -f      # follow, live
$ journalctl -u nginx --since "10 min ago"
```

- The same script, the same `echo`, three different destinations today: your terminal, a file, and the system journal. You never changed the script — you changed who was running it.
- From class 3 on, this is where you find out why your server will not start. Learn the `-u` habit now.

LAB02 · MANAGING PROCESSES

- Four parts, 28 steps. Parts 1-3 build the vocabulary; Part 4 is the point.
- `sudo` will ask for your password now — several times in Part 4. It is your own password, and nothing shows on screen as you type.
- Read the box near the top: from this lab on, your VM records how many minutes you were active. What it records, and what it does not, is written there in full — and the two files are yours to read.
- `int134 check lab02` as often as you like. It shows passes, not scores.

Ask early. Everything today is easier to see than to explain.